FASTAPI CRUD APPLICATION

Prepared by
Naveen Kumar S

1. Task Given

The task was to create a CRUD application using FastAPI with PostgreSQL connectivity and authentication. The application should allow a user to register, login, receive an access token and use authenticated APIs to create, read, update and delete tasks.

2. Application I Created

I created a FastAPI CRUD application and tested the complete workflow through the automatically generated Swagger UI. The application contains authentication, user and task endpoints.

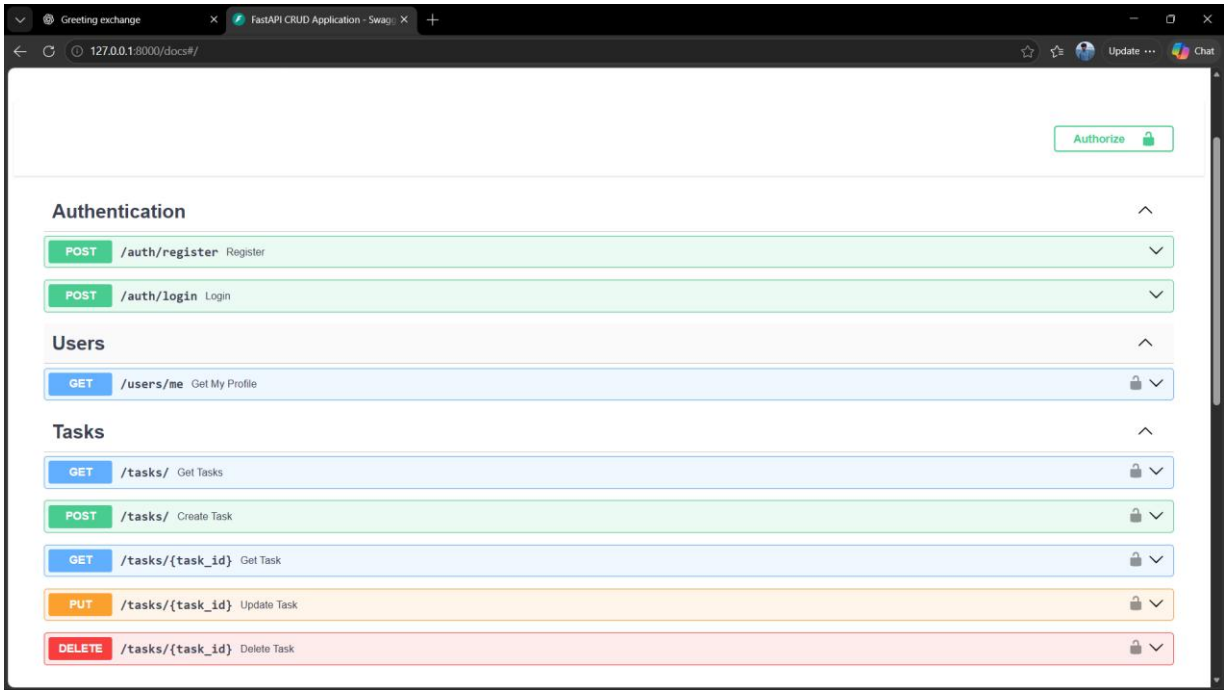


Figure 1 – Swagger UI showing the Authentication, Users and Tasks sections.

The main API sections available in the application are:

- Authentication – user registration and login.
- Users – getting the profile of the currently authenticated user.
- Tasks – creating, reading, updating and deleting tasks.

3. User Registration

The first step was to register a user. I used the POST /auth/register endpoint. The request contains the username, email and password of the new user.

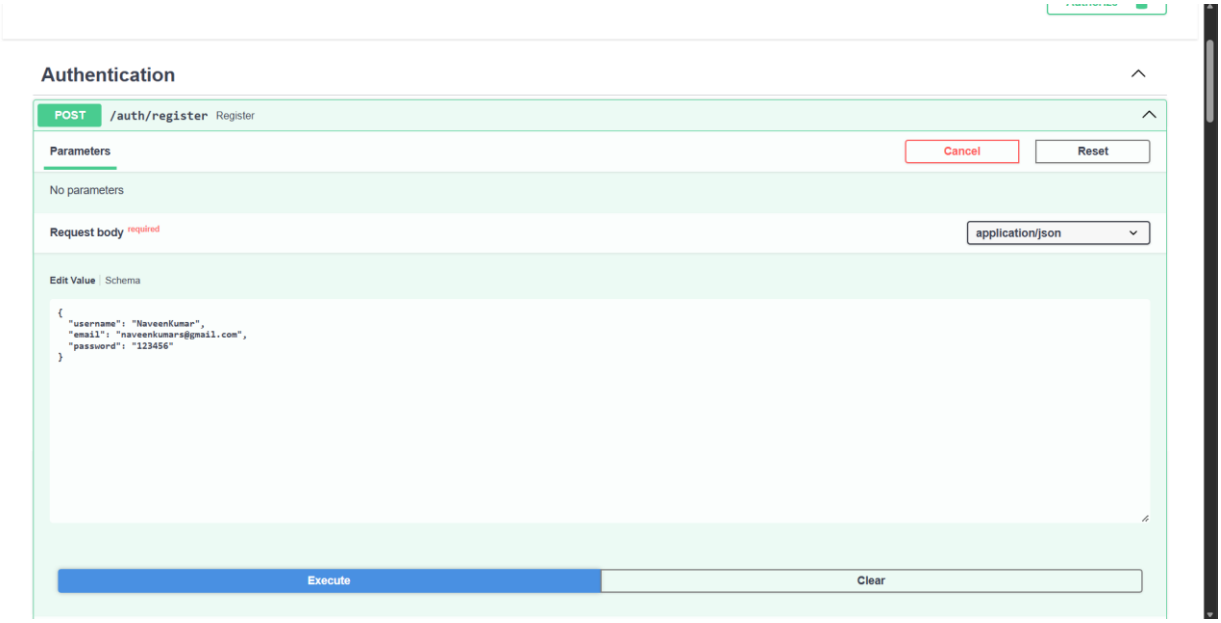


Figure 2 – User registration request entered in Swagger UI.

After executing the request, the API returned HTTP 201 Created. The response confirmed that the user was successfully created and returned the generated user ID, username and email.

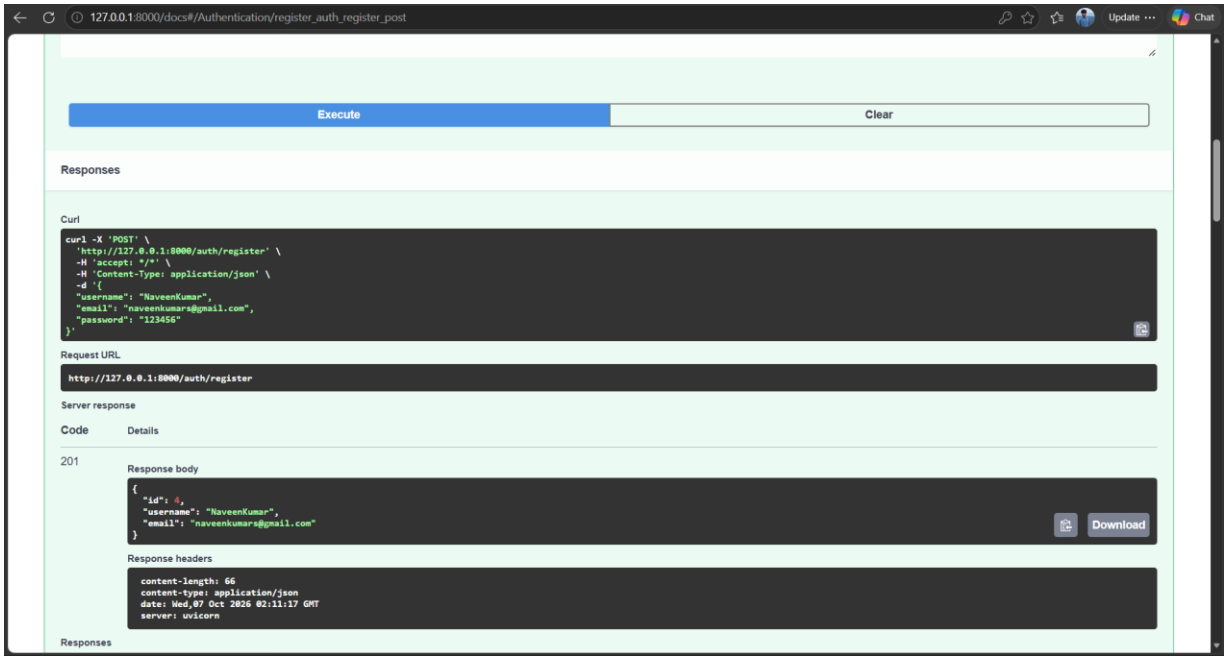


Figure 3 – Successful user registration response.

4. PostgreSQL – User Data

After registration, I verified the data in PostgreSQL using pgAdmin. The users table contains the user ID, username, email and hashed password. The password is stored as a hash instead of the original password.

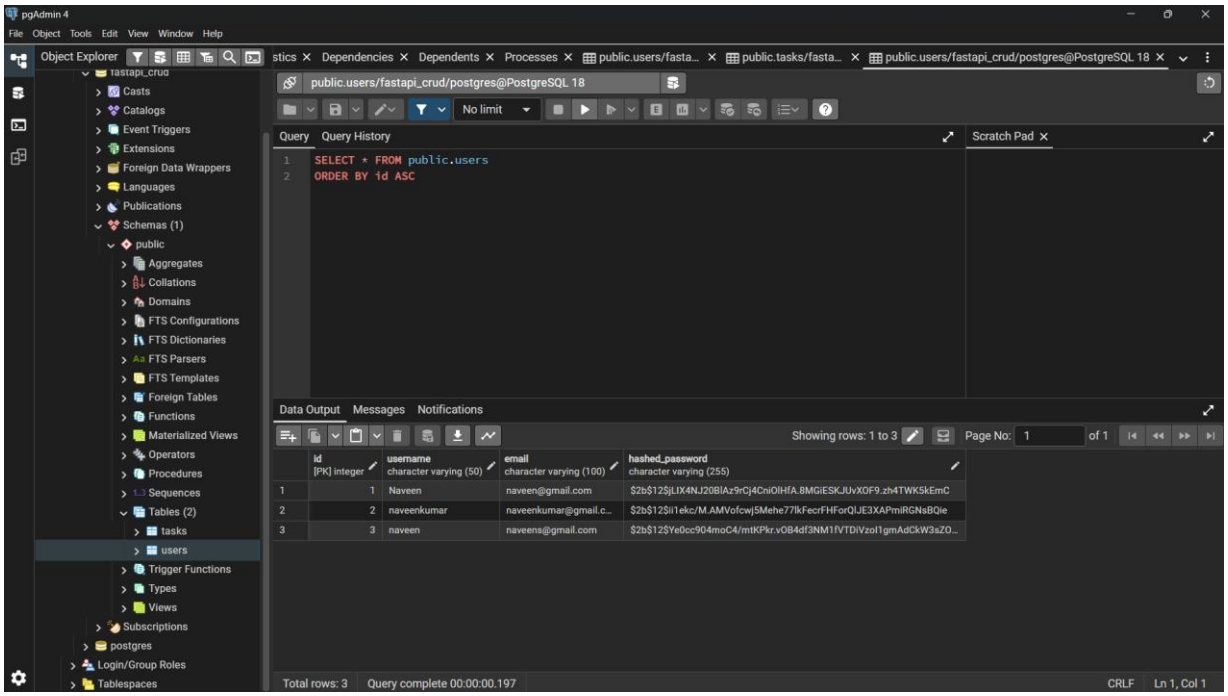


Figure 4 – PostgreSQL users table showing the registered users and hashed passwords.

5. User Login

After registration, I logged in through POST /auth/login using the registered username and password. The login request is used to verify the user's credentials.

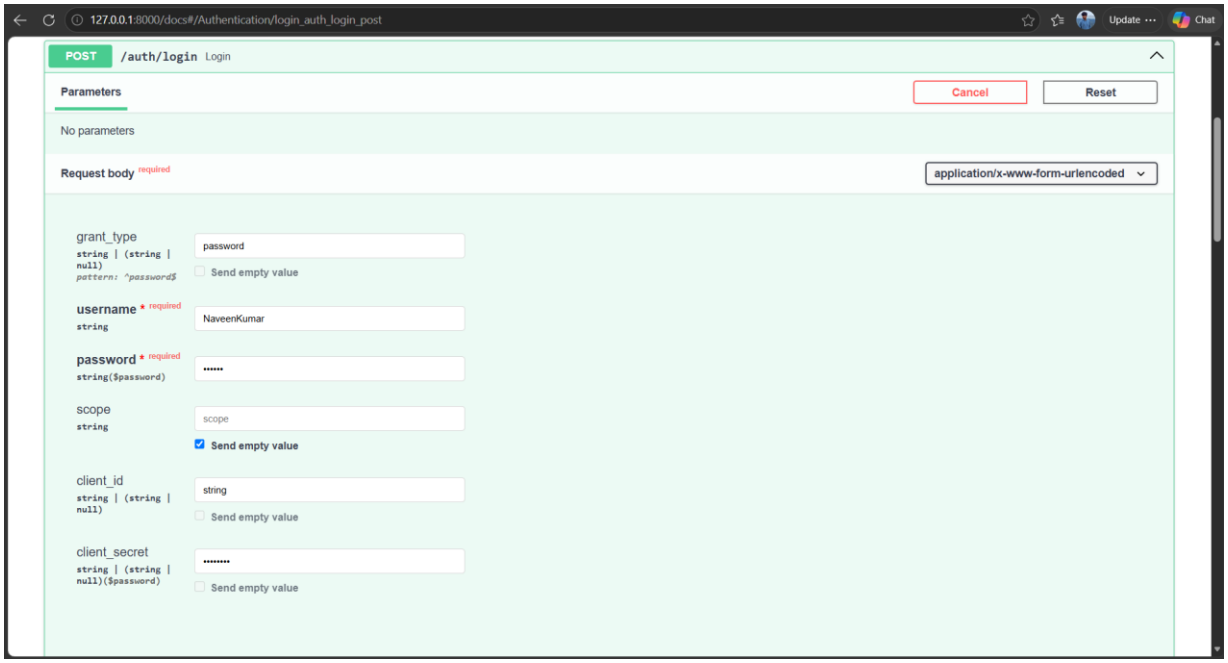


Figure 5 – Login request using username and password.

When the credentials were correct, the API returned HTTP 200 OK and generated an access token. The response contains the access_token and token_type.

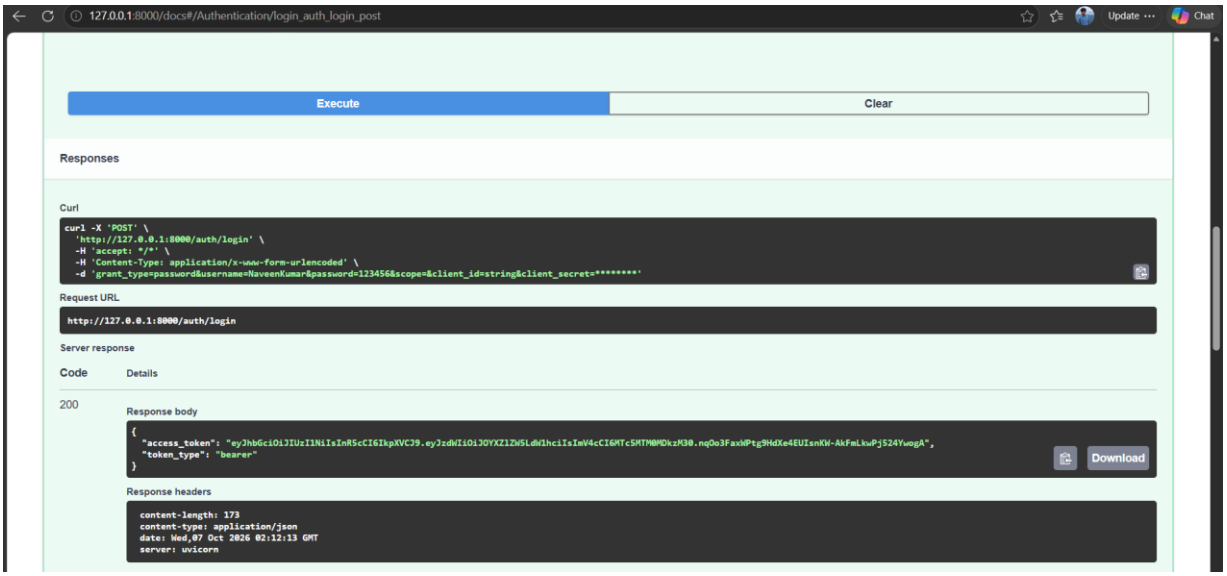


Figure 6 – Successful login response containing the JWT access token.

6. Understanding the Access Token

The password and access token have different purposes. The username and password are used during login to verify the user's identity. Once login is successful, the server generates a JWT access token.

For protected API requests, the application uses the access token instead of asking for the password again. The token is sent in the Authorization header using the Bearer scheme.

The general format is:

Authorization: Bearer <access_token>

7. Swagger Authorization

I used the Authorize button in Swagger UI and entered the access token obtained from the login response. This connects the JWT token to Swagger's protected API requests.

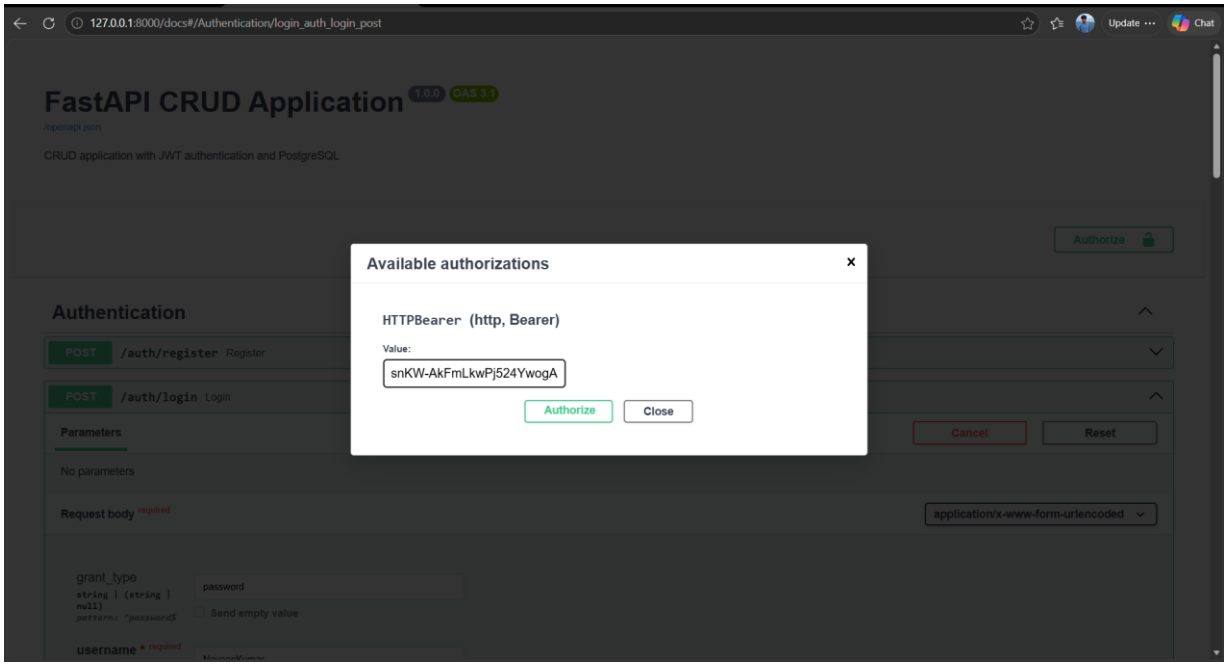


Figure 7 – Swagger Authorize dialog where the Bearer token is entered.

After authorization, Swagger displayed the authorization as active. From this point, protected endpoints automatically included the Bearer token in their requests.

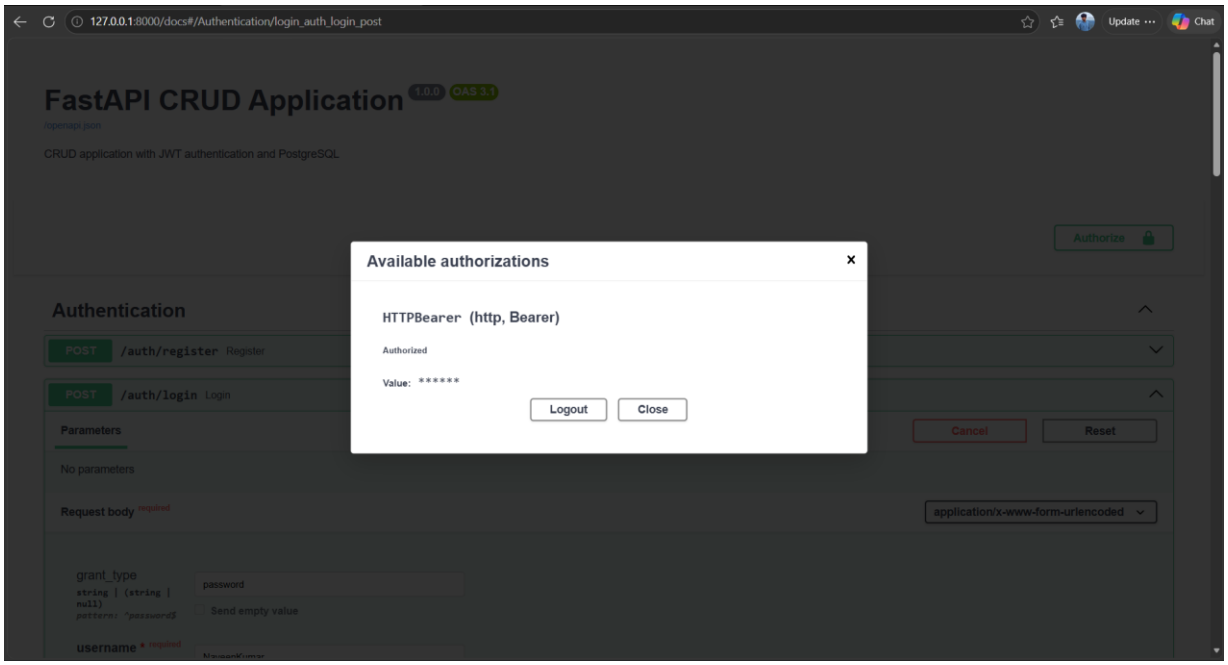


Figure 8 – Swagger showing that the Bearer authentication is authorized.

8. Get My Profile

I tested GET /users/me after authorization. This endpoint does not require the username and password again. It uses the JWT token to identify the current user.

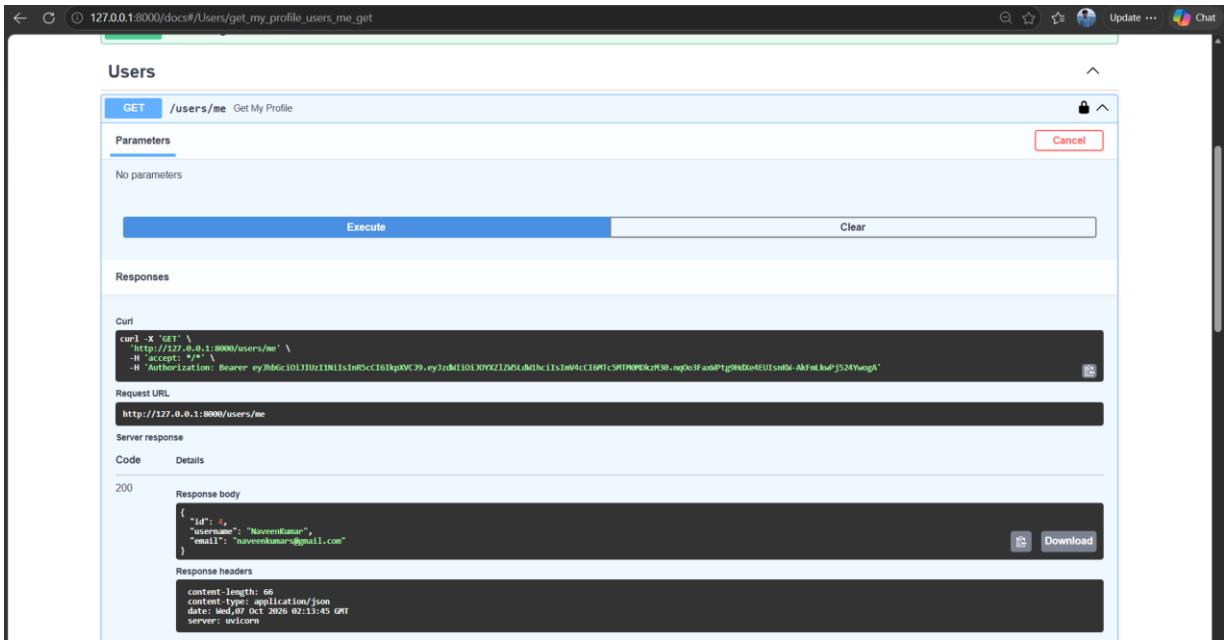


Figure 9 – GET /users/me response showing the authenticated user's profile.

This helped me understand the main purpose of the access token: after login, the token is used to prove that the request belongs to an authenticated user.

9. Create Task – CREATE

After authentication, I created a task using POST /tasks/. I entered the task title and description in the request body.

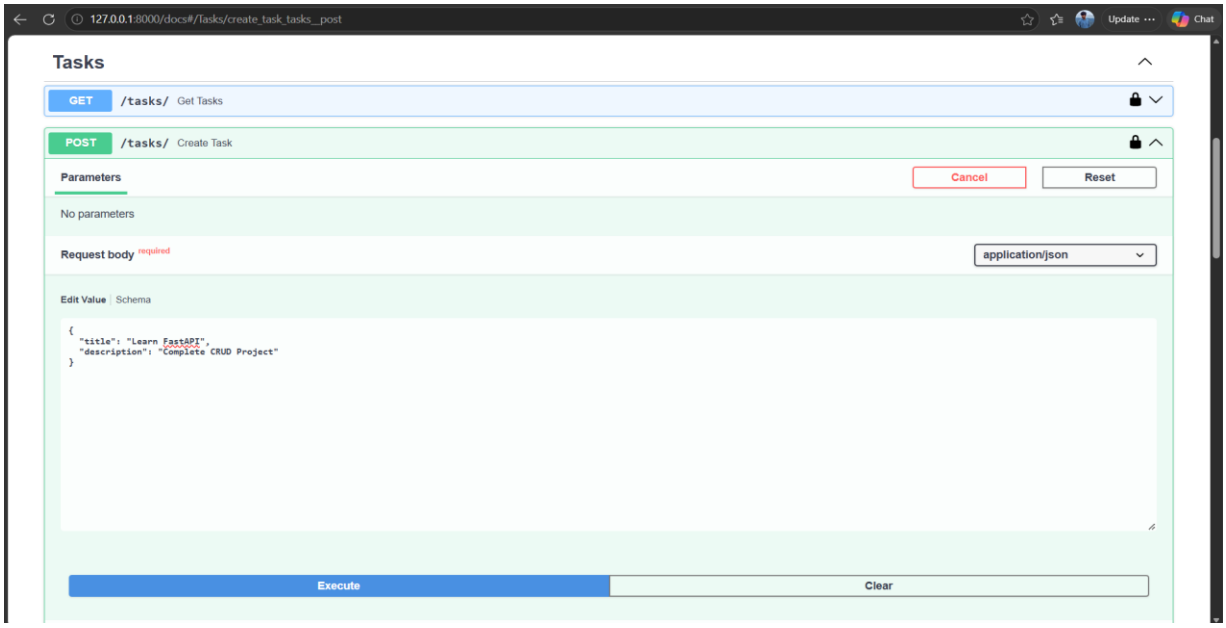


Figure 10 – Create Task request in Swagger UI.

After executing the request, the API returned HTTP 201 Created. The response contains the generated task ID, title, description, completed status and owner_id.

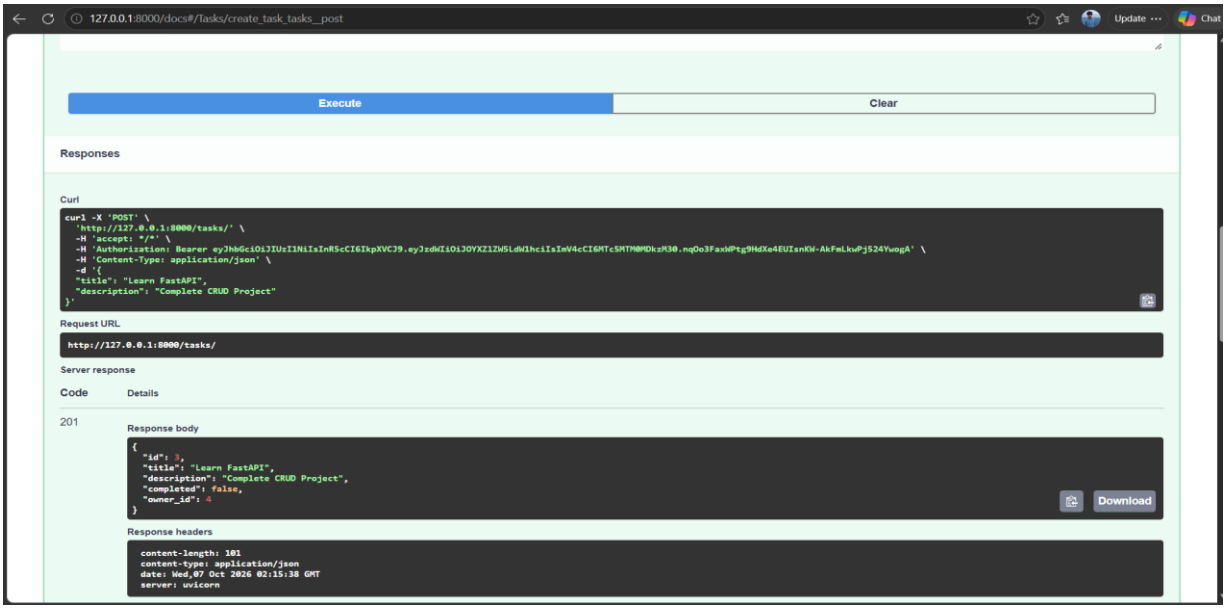


Figure 11 – Successful Create Task response.

The owner_id is assigned automatically from the currently authenticated user. I do not need to manually send the user ID while creating a task.

10. Read All Tasks – READ

I used the GET /tasks/ endpoint to retrieve tasks belonging to the authenticated user. The endpoint supports pagination through the skip and limit query parameters, allowing a specific number of tasks to be retrieved at a time. The request uses the Bearer token provided through Swagger authorization, and the application returns only tasks associated with the current user.

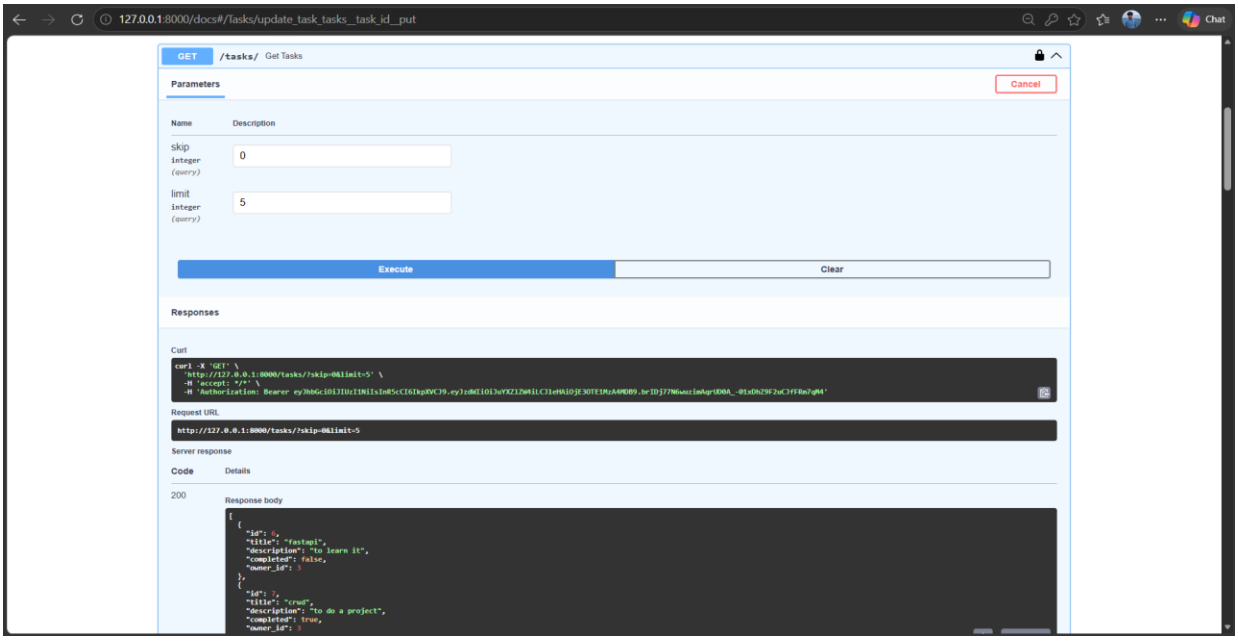


Figure 12 – GET /tasks/ response showing the user's tasks.

11. Read One Task

I used GET /tasks/{task_id} to retrieve one specific task. The task ID is provided as a path parameter. The API checks both the task ID and the authenticated user's ownership before returning the task.

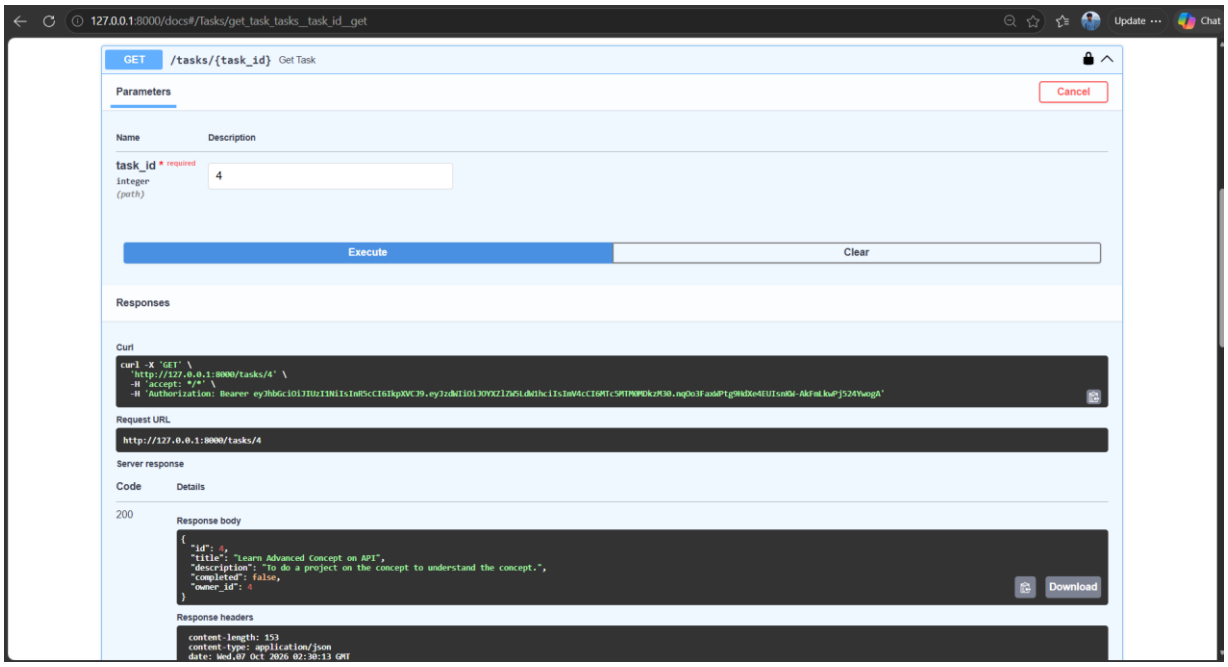


Figure 13 – GET /tasks/{task_id} response for a specific task.

12. Update Task – UPDATE

I used PUT /tasks/{task_id} to update an existing task. The task ID is supplied in the URL and the new task information is supplied in the request body.

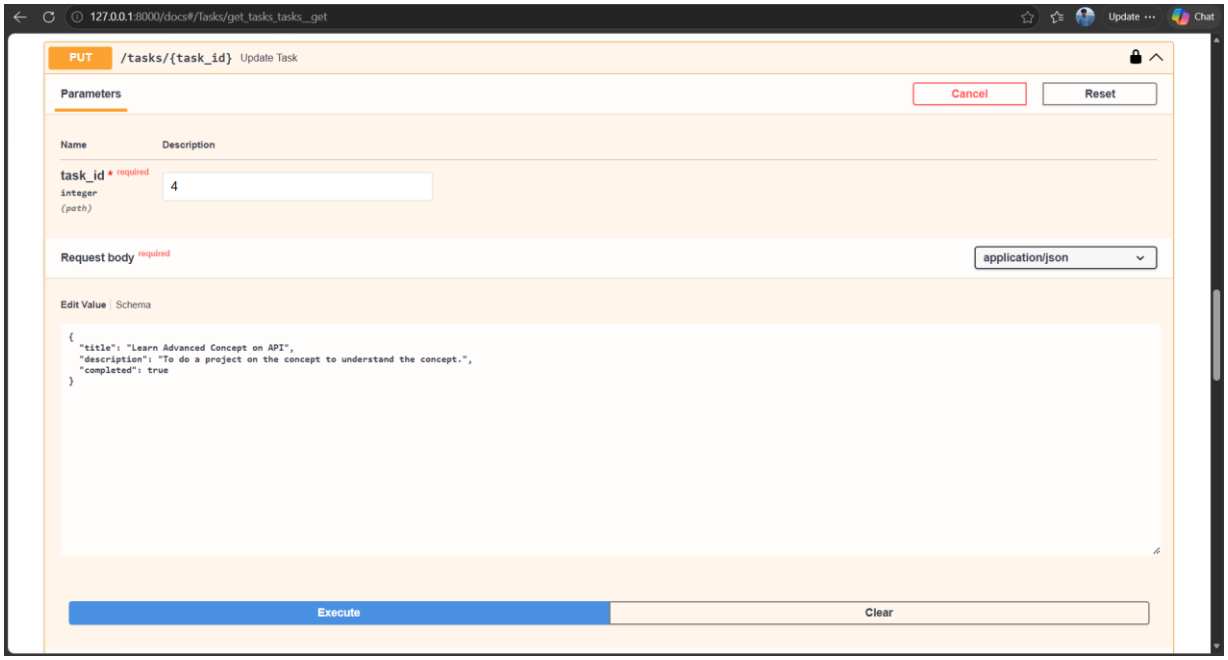


Figure 14 – Update Task request with task ID and updated values.

The API returned HTTP 200 OK and the response showed the updated task. In this test, the completed value was changed to true.

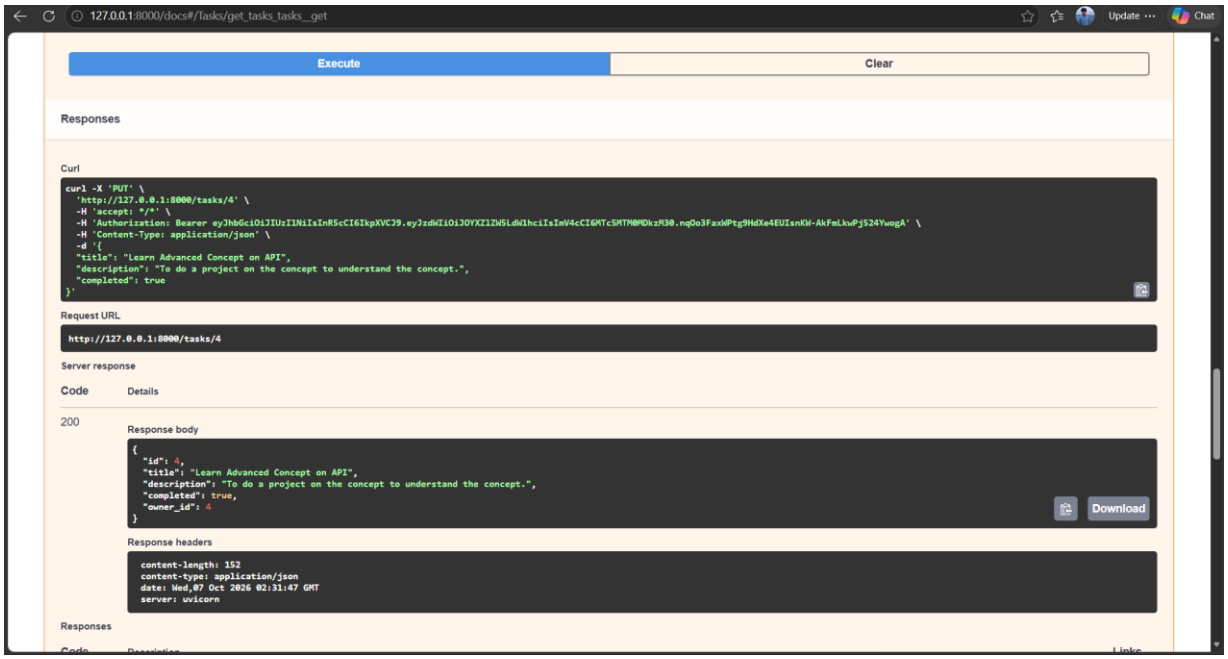


Figure 15 – Successful Update Task response.

13. Delete Task – DELETE

I used the DELETE /tasks/{task_id} endpoint to remove a task from the application. The task ID is provided as a path parameter. Before deleting the task, the application checks whether the task exists and belongs to the authenticated user.

After successful deletion, the API returns HTTP status code 204 No Content, indicating that the task was deleted successfully without returning a response body. If the task does not exist or does not belong to the current user, the API returns 404 Not Found.

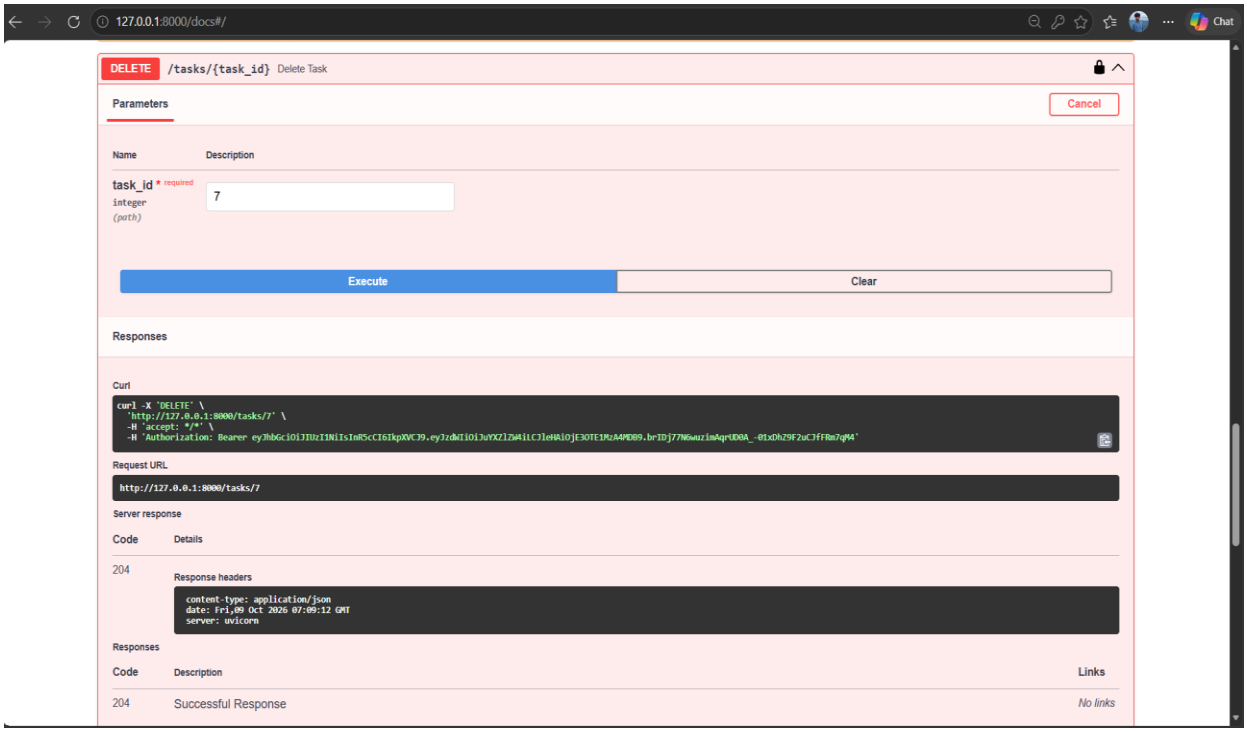


Figure 16 – Successful Delete Task response.

14. PostgreSQL – Task Data

I verified the task records in PostgreSQL using pgAdmin. The tasks table contains the task ID, title, description, completed status and owner_id.

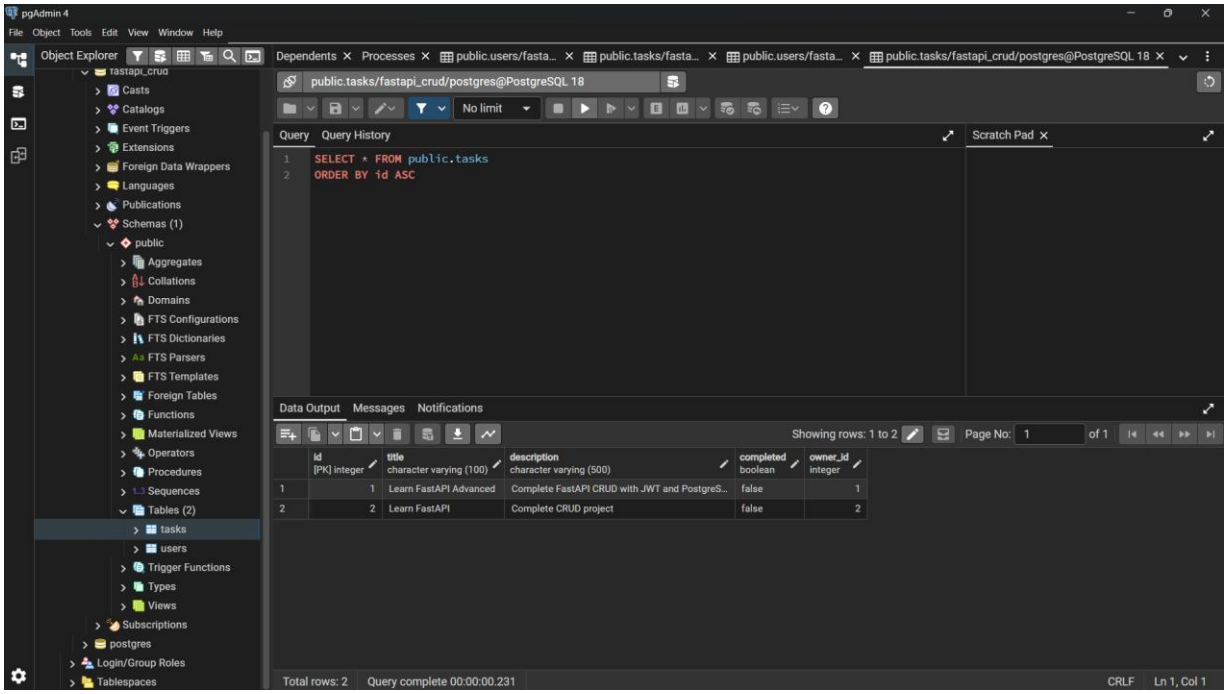


Figure 17 – PostgreSQL tasks table showing stored task records.

The owner_id creates the connection between a task and the user who owns it. This is how the application keeps each user's tasks associated with the correct user.

15. CRUD Operations Completed

Operation	Method	Endpoint	What I did
Create	POST	/tasks/	Created a new task
Read All	GET	/tasks/?skip=0&limit=5	Retrieved up to five tasks belonging to the authenticated user
Read One	GET	/tasks/{task_id}	Retrieved a specific task
Update	PUT	/tasks/{task_id}	Updated task details
Delete	DELETE	/tasks/{task_id}	Deleted a task

16. Complete Workflow I Followed

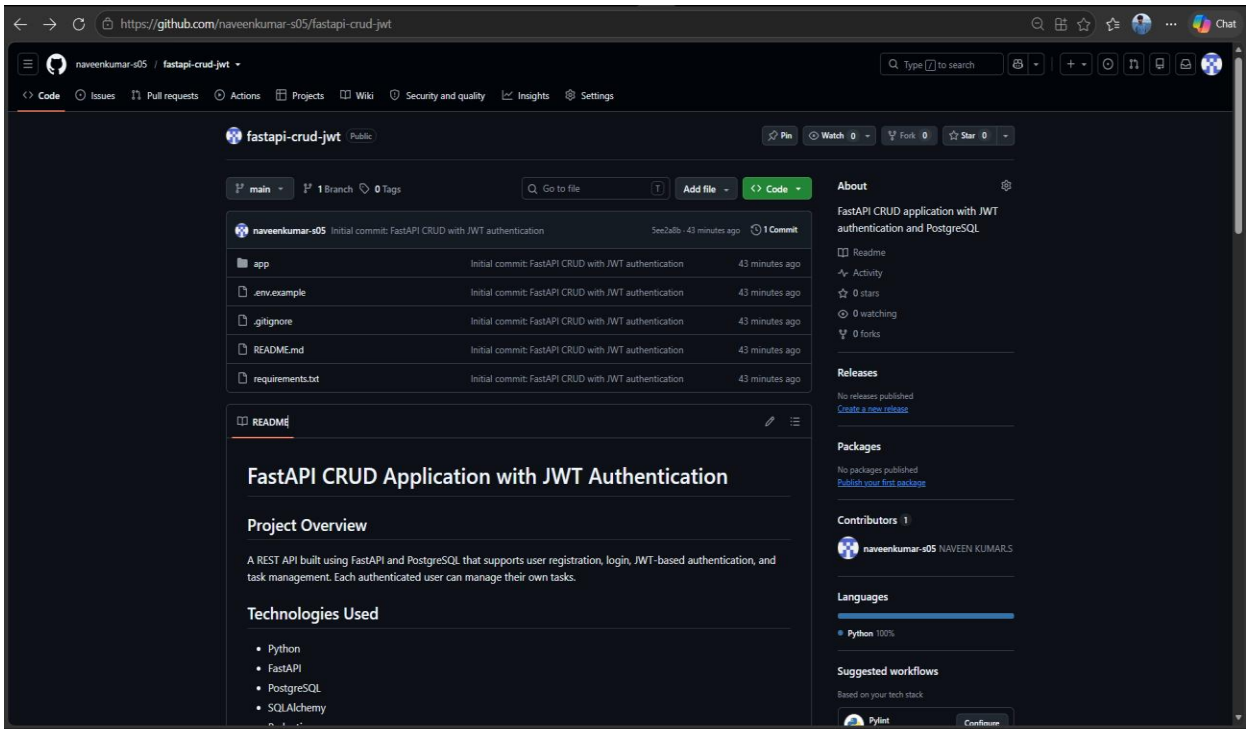
- Registered a user.
- Stored the registered user in PostgreSQL.
- Logged in using username and password.
- Received a JWT access token.
- Used the Authorize button to provide the Bearer token to Swagger.
- Accessed the protected user profile endpoint.
- Created tasks for the authenticated user.
- Read all tasks and a specific task.
- Used the skip and limit query parameters to retrieve tasks through pagination.
- Updated a task and changed its completion status.
- Deleted a task.
- Verified that the DELETE endpoint returned HTTP 204 No Content after successful deletion.
- Verified the final user and task records in PostgreSQL.

17. What I Learned

- How to create and organize a FastAPI backend application.
- How FastAPI routers are used to separate authentication, users and tasks.
- How to connect FastAPI with PostgreSQL using SQLAlchemy.
- How user registration and password hashing work.
- Why the password is used during login but the JWT access token is used for protected requests after login.
- How Bearer authentication works in Swagger UI.
- How JWT is decoded to identify the current user.
- How a task is connected to a user using owner_id and a foreign key.
- How to implement all CRUD operations using POST, GET, PUT and DELETE.
- How to test API endpoints and verify their HTTP responses using Swagger.
- How to verify that API data is actually stored in PostgreSQL using pgAdmin.
- How pagination works using the skip and limit query parameters.
- Why HTTP status code 204 No Content is used for a successful DELETE operation.

18. GitHub Repository and Submission

- The FastAPI CRUD application source code was uploaded to a public GitHub repository for version control and project submission.
- **GitHub Repository:** https://github.com/naveenkumar-s05/fastapi-crud-jwt
- The repository contains the application source code, including the authentication, database, models, schemas, dependencies and router modules. It also includes README.md with project information and setup instructions, requirements.txt for installing dependencies, and .env.example as a template for environment configuration.
- The actual .env file containing database credentials and secret configuration values is excluded from version control to protect sensitive information.
- Git was used to initialize the repository, track the project files, create a commit and push the code to the public GitHub repository.



19. Final Result

The FastAPI CRUD application was implemented using Python, FastAPI, PostgreSQL, SQLAlchemy and JWT authentication. The application supports user registration, login, token-based authentication, protected user access and task creation, retrieval, updating and deletion.

Pagination was implemented in the task listing endpoint using the skip and limit query parameters. The DELETE endpoint was updated and tested to return HTTP 204 No Content after successful deletion. The application data was verified in PostgreSQL using pgAdmin.

The source code was uploaded to a public GitHub repository with a README.md file, requirements.txt and .env.example. The actual environment file containing database credentials is excluded from version control.